At the most basic level, a website consists of an HTML document. The browser that you use to view the website is a program that interprets HTML and CSS and renders the style, content, and structure into the page that you see.

In addition to parsing the style and structure of the HTML and CSS, the browser creates a representation of the document known as the Document Object Model. This model allows JavaScript to access the text content and elements of the website document as objects.

JavaScript is an interactive language, and it is easier to understand new concepts by doing. Let's create a very basic website. Create an `index.html` file and save it in a new project directory.

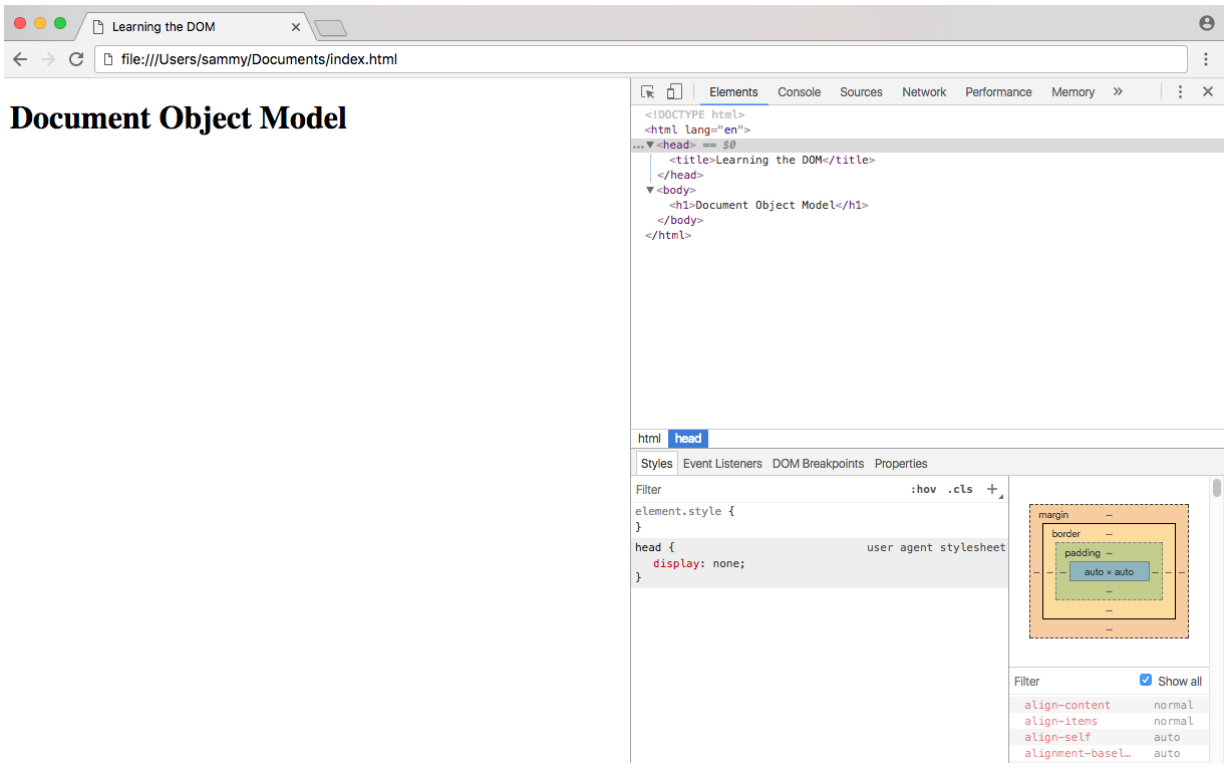
index.html

```html
<!DOCTYPE html>
<html lang="en">

  <head>
    <title>Learning the DOM</title>
  </head>

  <body>
    <h1>Document Object Model</h1>
  </body>

</html>
```

This code is the familiar HTML source of a new website skeleton. It contains the absolute most essential aspects of a website document — a doctype, and an `html` tag with the `head` and `body` nested inside.

For our purposes, we'll be using the Chrome browser, but you can receive similar output from other modern browser. Within Chrome, open up `index.html`. You'll see a plain website with our heading saying "Document Object Model". Right click anywhere on the page and select "Inspect". This will open up Developer Tools.

Under the Elements tab, you'll see the DOM.



DOM Example

In this case, when expanded, it looks exactly the same as the HTML source code we just wrote — a doctype, and the few other HTML tags that

we added. Hovering over each element will highlight the respective element in the rendered website. Little arrows to the left of the HTML elements allow you to toggle the view of nested elements.

The Document Object

The `document` object is a built-in object that has many properties and methods that we can use to access and modify websites. In order to understand how to work with the DOM, you must understand how objects work in JavaScript. Review Understanding Objects in JavaScript if you don't feel comfortable with the concept of objects.

In Developer Tools on index.html, move to the Console tab. Type `document` into the console and press `ENTER`. You will see that what is output is the same as what you see in the Elements tab.

```
document;
```

Output

```
#document
<!DOCTYPE html>
<html lang="en">

  <head>
    <title>Learning the DOM</title>
  </head>

  <body>
    <h1>Document Object Model</h1>
  </body>

</html>
```

Typing `document` and otherwise working directly in the console is not something that you'll generally ever do outside of debugging, but it helps solidify exactly what the `document` object is and how to modify it, as we will discover below.

What is the Difference Between the DOM and HTML Source Code?

Currently, with this example, it seems that HTML source code and the DOM are the exact same thing. There are two instances in which the browser-generated DOM will be different than HTML source code:

- The DOM is modified by client-side JavaScript
- The browser automatically fixes errors in the source code

Let's demonstrate how the DOM can be modified by client-side JavaScript. Type the following into the console:

```
document.body;
```

The console will respond with this output:

Output

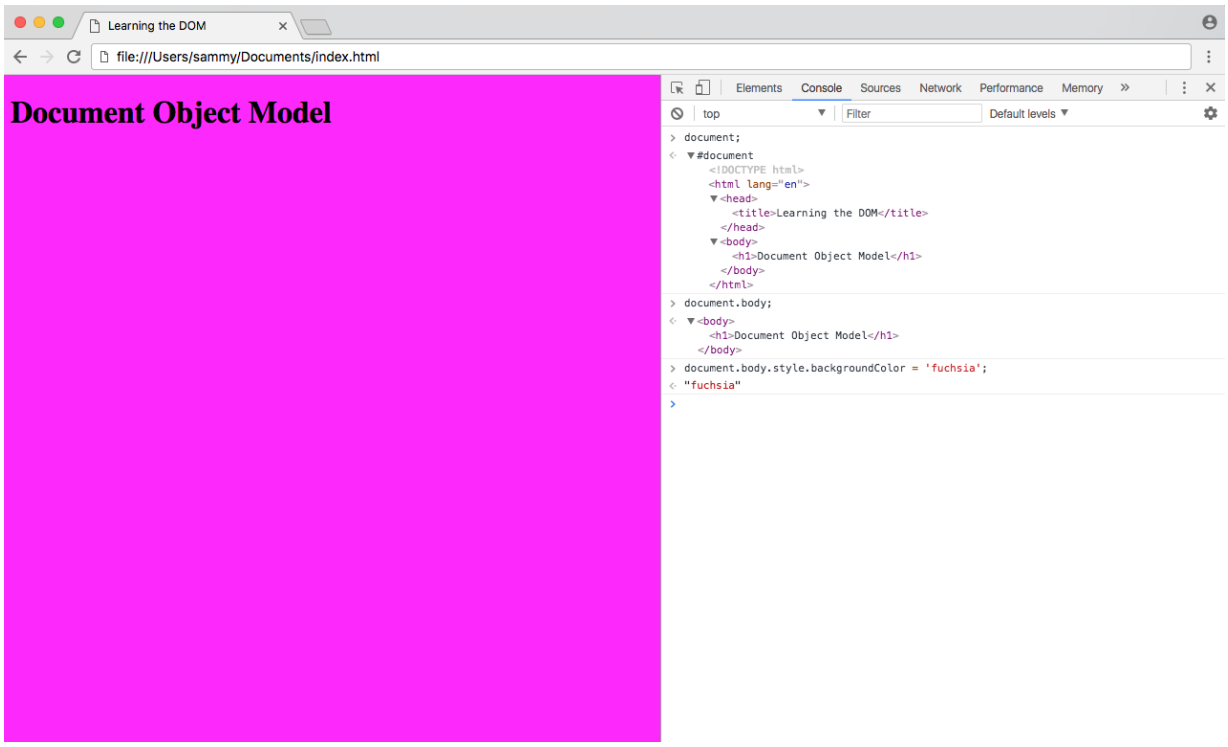
```
<body>
  <h1>Document Object Model</h1>
</body>
```

`document` is an object, `body` is a property of that object that we have accessed with dot notation. Submitting `document.body` to the console outputs the `body` element and everything inside of it.

In the console, we can change some of the live properties of the `body` object on this website. We'll edit the `style` attribute, changing the background color to `fuchsia`. Type this into the console:

```
document.body.style.backgroundColor = 'fuchsia';
```

After typing and submitting the above code, you'll see the live update to the site, as the background color changes.



DOM Live Modification

Switching to the Elements tab, or typing `document.body` into the console again, you will see that the DOM has changed.

Output

```
<body style="background-color: fuchsia;">
  <h1>Document Object Model</h1>
</body>
```

Note: In order to change the `background-color` CSS property, we had to type `backgroundColor` in the JavaScript. Any hyphenated CSS property will be written in camelCase in JavaScript.

The JavaScript code we typed, assigning `fuchsia` to the background color of the `body`, is now a part of the DOM.

However, right click on the page and select "View Page Source". You will notice that the source of the website does not contain the new style attribute we added via JavaScript. The source of a website will not change and will never be affected by client-side JavaScript. If you refresh the page, the new code we added in the console will disappear.

The other instance in which the DOM might have a different output than HTML source code is when there are errors in the source code. One common example of this is the `table` tag — a `tbody` tag is required inside a `table`, but developers often fail to include it in their HTML. The browser will automatically correct the error and add the `tbody`, modifying the DOM. The DOM will also fix tags that have not been closed.

Conclusion

In this tutorial, we defined the DOM, accessed the `document` object, used JavaScript and the console to update a property of the `document` object, and went over the difference between HTML source code and the DOM.

For more in-depth information on the DOM, review the Document Object Model (DOM) page on the Mozilla Developer Network.

In the next tutorial, we will review important HTML terminology, learn about the DOM tree, discover what nodes are, learn about the most common types of nodes, and begin creating interactive scripts with JavaScript.

Understanding the DOM Tree and Nodes

Written by Tania Rascia

The DOM is often referred to as the DOM tree, and consists of a tree of objects called nodes. In the Introduction to the DOM, we went over what the Document Object Model (DOM) is, how to access the `document` object and modify its properties with the console, and the difference between HTML source code and the DOM.

In this tutorial, we will review HTML terminology, which is essential to working with JavaScript and the DOM, and we will learn about the DOM tree, what nodes are, and how to identify the most common node types. Finally, we will move beyond the console and create a JavaScript program to interactively modify the DOM.

HTML Terminology

Understanding HTML and JavaScript terminology is essential to understanding how to work with the DOM. Let's briefly review some HTML terminology.

To begin, let's take a look at this HTML element.

```
<a href="index.html">Home</a>
```

Here we have an anchor element, which is a link to `index.html`.

- `a` is the tag
- `href` is the attribute

- `index.html` is the attribute value
- `Home` is the text.

Everything between the opening and closing tag combined make the entire HTML element.

We'll be working with the `index.html` from the previous tutorial:

index.html

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <title>Learning the DOM</title>
  </head>

  <body>
    <h1>Document Object Model</h1>
  </body>

</html>
```

The simplest way to access an element with JavaScript is by the `id` attribute. Let's add the link we have above into our `index.html` file with an `id` of `nav`.

index.html

```
...
<body>
  <h1>Document Object Model</h1>
  <^><a id="nav" href="index.html">Home</a><^>
</body>
...
```

Load or reload the page in your browser window and look at the DOM to ensure that the code has been updated.

We're going to use the `getElementById()` method to access the entire element. In the console, type the following:

```
document.getElementById('nav');
```

Output

```
<a id="nav" href="index.html">Home</a>
```

We have retrieved the entire element using `getElementById()`. Now, instead of typing that object and method every time we want to access the `nav` link, we can place the element into a variable to work with it more easily.

```
let navLink = document.getElementById('nav');
```

The `navLink` variable contains our anchor element. From here, we can easily modify attributes and values. For example, we can change where the

link goes by changing the `href` attribute:

```
navLink.href = 'https://www.wikipedia.org';
```

We can also change the text content by reassigning the `textContent` property:

```
navLink.textContent = 'Navigate to Wikipedia';
```

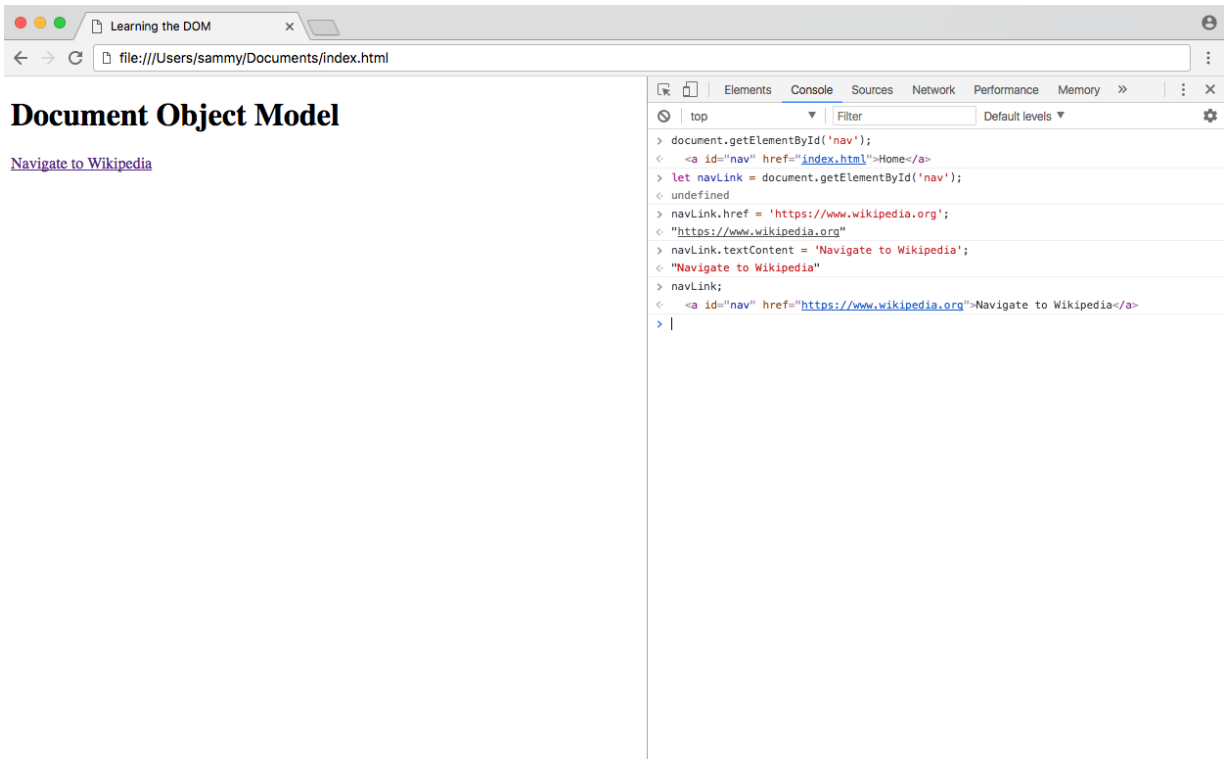
Now when we view our element, either in the console or by checking the Elements tag, we can see how the element has been updated.

```
navLink;
```

Output

```
<a id="nav"
href="https://www.wikipedia.org/">Navigate to
Wikipedia</a>
```

This is also reflected on the front-end of the website.



Updated Link via DOM

Refreshing the page will revert everything back to their original values.

At this point, you should understand how to use a `document` method to access an element, how to assign an element to a variable, and how to modify properties and values in the element.

The DOM Tree and Nodes

All items in the DOM are defined as nodes. There are many types of nodes, but there are three main ones that we work with most often:

- Element nodes
- Text nodes
- Comment nodes

When an HTML element is an item in the DOM, it is referred to as an element node. Any lone text outside of an element is a text node, and an HTML comment is a comment node. In addition to these three node types, the `document` itself is a document node, which is the root of all other nodes.

The DOM consists of a tree structure of nested nodes, which is often referred to as the DOM tree. You may be familiar with an ancestral family tree, which consists of parents, children, and siblings. The nodes in the DOM are also referred to as parents, children, and siblings, depending on their relation to other nodes.

To demonstrate, create a `nodes.html` file. We'll add text, comment, and element nodes.

nodes.html

```
<!DOCTYPE html>
<html>

  <head>
    <title>Learning About Nodes</title>
  </head>

  <body>
    <h1>An element node</h1>
    <!-- a comment node -->
    A text node.
  </body>

</html>
```

The `html` element node is the parent node. `head` and `body` are siblings, children of `html`. `body` contains three child nodes, which are all siblings — the type of node does not change the level at which it is nested.

Note: When working with an HTML-generated DOM, the indentation of the HTML source code will create many empty text nodes, which won't be visible from the DevTools Elements tab. Read about [Whitespace in the DOM]

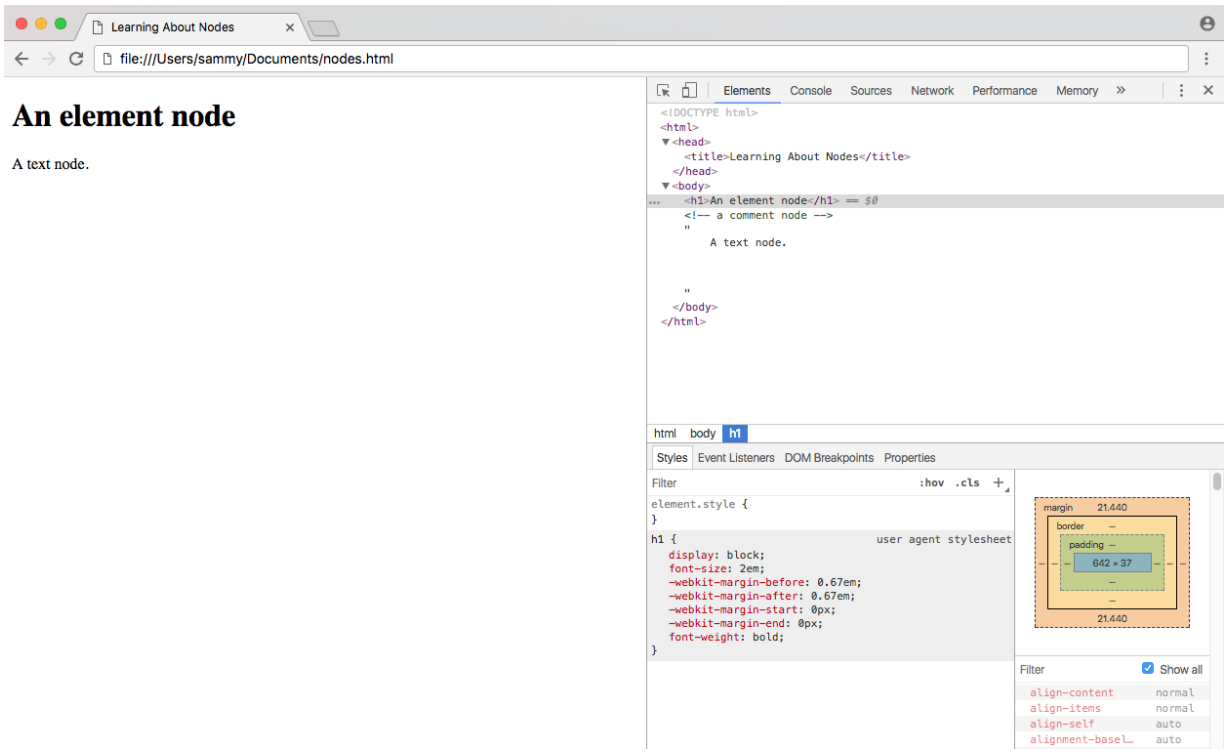
Identifying Node Type

Every node in a document has a node type, which is accessed through the `nodeType` property. The Mozilla Developer Network has an up-to-date list of all node type constants. Below is a chart of the most common node types that we are working with in this tutorial.

Node Type	Value	Example
`ELEMENT_NODE`	1	The `<body>` element
`TEXT_NODE`	3	Text that is not part of an element
`COMMENT_NODE`	8	`<!-- an HTML comment -->`

In the Elements tab of Developer Tools, you may notice that whenever you click on and highlight any line in the DOM the value of `== $0` will appear next to it. This is a very handy way to access the currently active element in Developer Tools by typing `$0`.

In the console of nodes.html, click on the first element in the `body`, which is an `h1` element.



DOM Node Type

In the console, get the node type of the currently selected node with the `nodeType` property.

```
$0.nodeType;
```

Output

```
1
```

With the `h1` element selected, you would see `1` as the output, which we can see correlates to `ELEMENT_NODE`. Do the same for the text and the comment, and they will output `3` and `8` respectively.

When you know how to access an element, you can see the node type without highlighting the element in the DOM.

```
document.body.nodeType;
```

Output

```
1
```

In addition to `nodeType`, you can also use the `nodeValue` property to get the value of a text or comment node, and `nodeName` to get the tag name of an element.

Modifying the DOM with Events

Up until now, we've only seen how to modify the DOM in the console, which we have seen is temporary; every time the page is refreshed, the changes are lost. In the Introduction to the DOM tutorial, we used the console to update the background color of the body. We can combine what we've learned throughout this tutorial to create an interactive button that does this when clicked.

Let's go back to our `index.html` file and add a `button` element with an `id`. We'll also add a link to a new file in a new `js` directory `js/scripts.js`.

index.html

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <title>Learning the DOM</title>
  </head>

  <body>
    <h1>Document Object Model</h1>
    <button id="changeBackground">Change
Background Color</button>

    <script src="scripts.js"></script>
  </body>

</html>
```

An event in JavaScript is an action the user has taken. When the user hovers their mouse over an element, or clicks on an element, or presses a specific key on the keyboard, these are all types of events. In this particular case, we want our button to listen and be ready to perform an action when the user clicks on it. We can do this by adding an event listener to our button.

Create `scripts.js` and save it in the new `js` directory. Within the file, we'll first find the `button` element and assign it to a variable.

js/scripts.js

```
let button =
document.getElementById('changeBackground');
```

Using the `addEventListener()` method, we will tell the button to listen for a click, and perform a function once clicked.

js/scripts.js

```
...
button.addEventListener('click', () => {
  // action will go here
});
```

Finally, inside of the function, we will write the same code from the previous tutorial to change the background color to `fuchsia`.

js/scripts.js

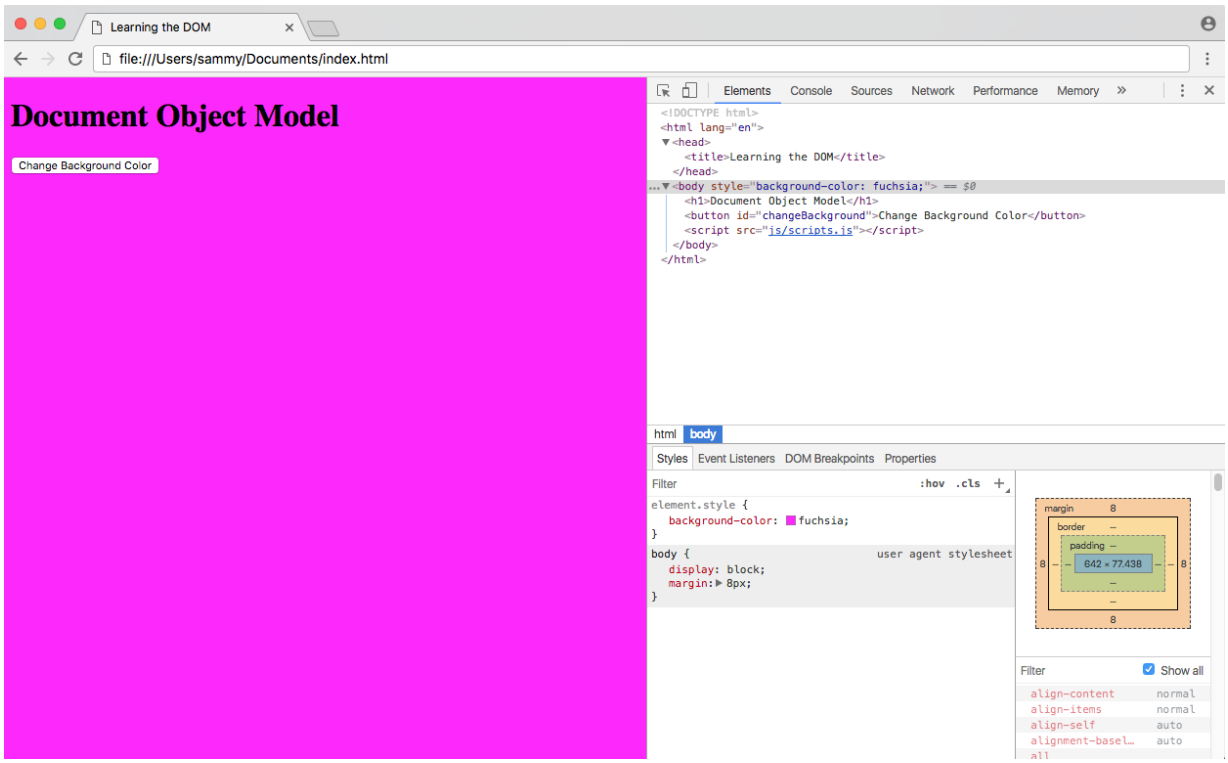
```
...
document.body.style.backgroundColor = 'fuchsia';
```

Here is our entire script:

js/scripts.js

```
let button =
document.getElementById('changeBackground');

button.addEventListener('click', () => {
  document.body.style.backgroundColor = 'fuchsia';
});
```

Once you save this file, refresh `index.html` in the browser. Click the button, and the event will fire.



Modify Background with Events

The background color of the page has changed to fuchsia due to the JavaScript event.

Conclusion

In this tutorial, we reviewed terminology that will allow us to understand and modify the DOM. We learned how the DOM is structured as a tree of nodes that will usually be HTML elements, text, or comments, and we created a script that would allow a user to modify a website without having to manually type code into the developer console.

How To Access Elements in the DOM

Written by Tania Rascia

In Understanding the DOM Tree and Nodes, we went over how the DOM is structured as a tree of objects called nodes, and that nodes can be text, comments, or elements. Usually when we access content in the DOM, it will be through an HTML element node.

In order to be proficient at accessing elements in the DOM, it is necessary to have a working knowledge of CSS selectors, syntax and terminology as well as an understanding of HTML elements. In this tutorial, we will go over several ways to access elements in the DOM: by ID, class, tag, and query selectors.

Overview

Here is a table overview of the five methods we will cover in this tutorial.

Gets	Selector Syntax	Method
ID	`#demo`	`getElementById()`
Class	`.demo`	`getElementsByClassName()`
Tag	`demo`	`getElementsByTagName()`
Selector (single)		`querySelector()`
Selector (all)		`querySelectorAll()`

It is important when studying the DOM to type the examples on your own computer to ensure that you are understanding and retaining the information you learn.

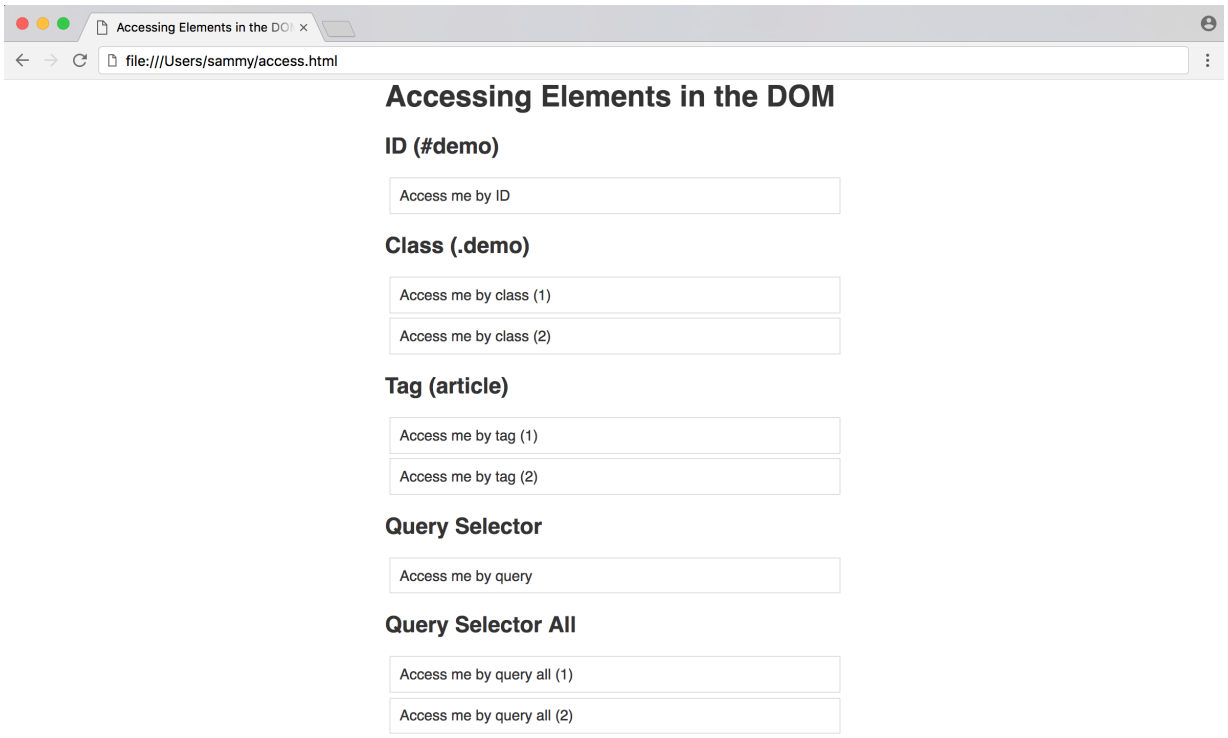
You can save this HTML file, `access.html`, to your own project to work through the examples along with this article. If you are unsure how to work with JavaScript and HTML locally, review our How To Add JavaScript to HTML tutorial.

access.html

```html
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Accessing Elements in the DOM</title>

  <style>
    html { font-family: sans-serif; color: #333; }
    body { max-width: 500px; margin: 0 auto; padding: 0 15px; }
    div, article { padding: 10px; margin: 5px; border: 1px solid #dedede; }
  </style>

</head>

<body>

  <h1>Accessing Elements in the DOM</h1>

  <h2>ID (#demo)</h2>
  <div id="demo">Access me by ID</div>
```

```
  <h2>Class (.demo)</h2>
  <div class="demo">Access me by class (1)</div>
  <div class="demo">Access me by class (2)</div>

  <h2>Tag (article)</h2>
  <article>Access me by tag (1)</article>
  <article>Access me by tag (2)</article>

  <h2>Query Selector</h2>
  <div id="demo-query">Access me by query</div>

  <h2>Query Selector All</h2>
  <div class="demo-query-all">Access me by query
all (1)</div>
  <div class="demo-query-all">Access me by query
all (2)</div>

</body>

</html>
```

In this HTML file, we have many elements that we will access with different `document` methods. When we render the file in a browser, it will look similar to this:



Browser rendering of access.html page

We'll be using the different methods that we outlined in the Overview above to access the available elements in the file.

Accessing Elements by ID

The easiest way to access a single element in the DOM is by its unique ID. We can grab an element by ID with the `getElementById()` method of the `document` object.

```
document.getElementById();
```

In order to be accessed by ID, the HTML element must have an `id` attribute. We have a `div` element with an ID of `demo`.

```
<div id="demo">Access me by ID</div>
```

In the Console, let's get the element and assign it to the `demoId` variable.

```
const demoId = document.getElementById('demo');
```

Logging `demoId` to the console will return our entire HTML element.

```
console.log(demoId);
```

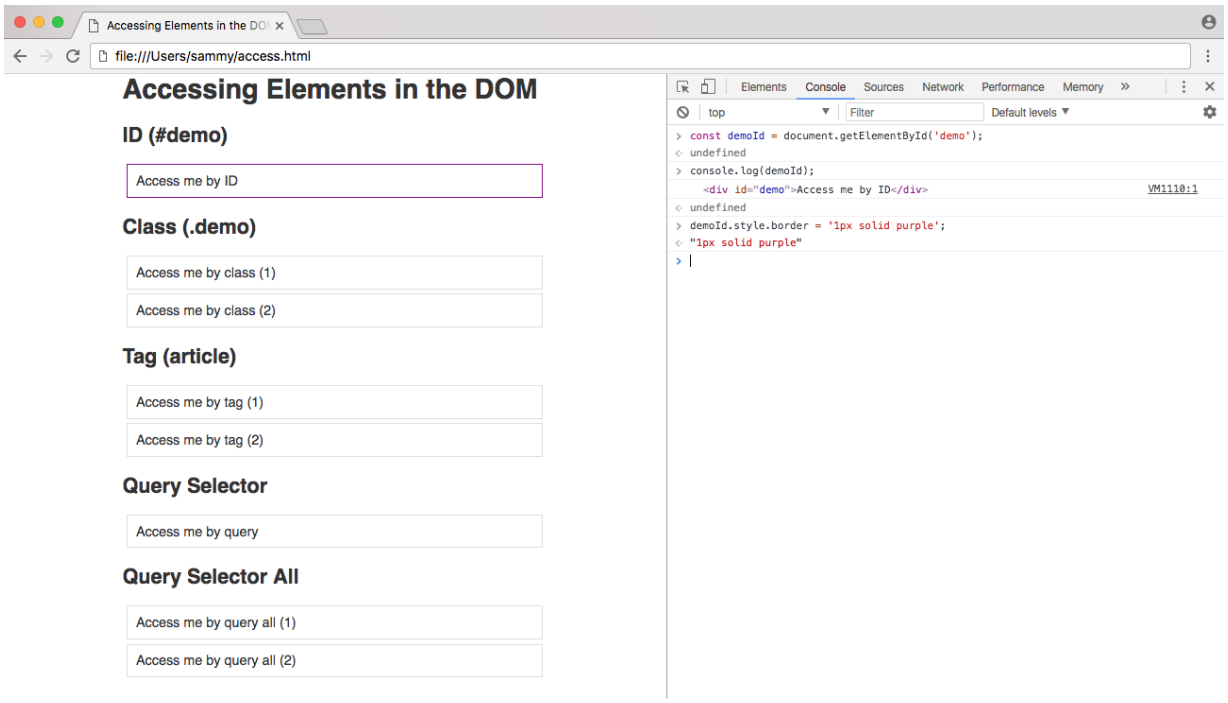
Output

```
<div id="demo">Access me by ID</div>
```

We can be sure we're accessing the correct element by changing the `border` property to `purple`.

```
demoId.style.border = '1px solid purple';
```

Once we do so, our live page will look like this:



Browser rendering of ID element styling

Accessing an element by ID is an effective way to get an element quickly in the DOM. However, it has drawbacks; an ID must always be unique to the page, and therefore you will only ever be able to access a single element at a time with the `getElementById()` method. If you wanted to add a function to many elements throughout the page, your code would quickly become repititious.

Accessing Elements by Class

The class attribute is used to access one or more specific elements in the DOM. We can get all the elements with a given class name with the `getElementsByClassName()` method.

```
document.getElementsByClassName();
```

Now we want to access more than one element, and in our example we have two elements with a `demo` class.

```
<div class="demo">Access me by class (1)</div>
<div class="demo">Access me by class (2)</div>
```

Let's access our elements in the Console and put them in a variable called `demoClass`.

```
const demoClass =
document.getElementsByClassName('demo');
```

At this point, you might think you can modify the elements the same way you did with the ID example. However, if we try to run the following code and change the `border` property of the class demo elements to orange, we will get an error.

```
demoClass.style.border = '1px solid orange';
```

Output

```
Uncaught TypeError: Cannot set property 'border'
of undefined
```

The reason this doesn't work is because instead of just getting one element, we have an array-like object of elements.

```
console.log(demoClass);
```

Output

```
(2) [div.demo, div.demo]
```

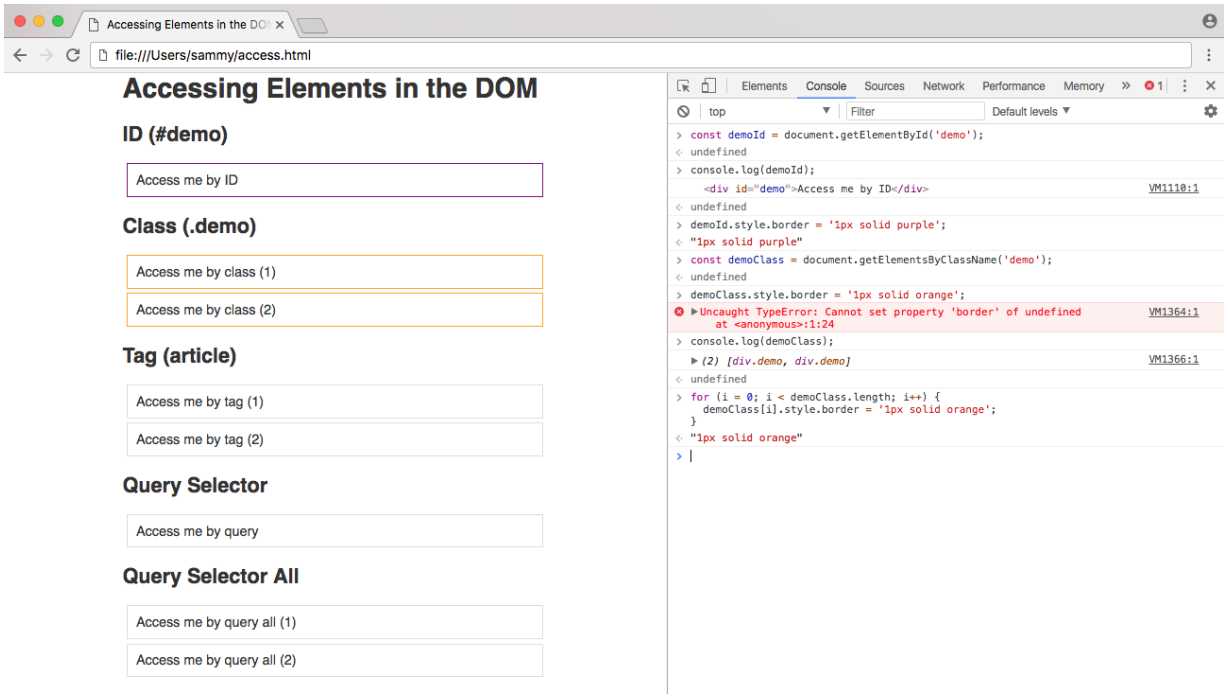
JavaScript arrays must be accessed with an index number. We can therefore change the first element of this array by using an index of `0`.

```
demoClass[0].style.border = '1px solid orange';
```

Generally when accessing elements by class, we want to apply a change to all the elements in the document with that particular class, not just one. We can do this by creating a `for` loop, and looping through every item in the array.

```
for (i = 0; i < demoClass.length; i++) {
  demoClass[i].style.border = '1px solid orange';
}
```

When we run this code, our live page will be rendered like this:



Browser rendering of class element styling

We have now selected every element on the page that has a `demo` class, and changed the `border` property to `orange`.

Accessing Elements by Tag

A less specific way to access multiple elements on the page would be by its HTML tag name. We access an element by tag with the `getElementsByTagName()` method.

```
document.getElementsByTagName();
```

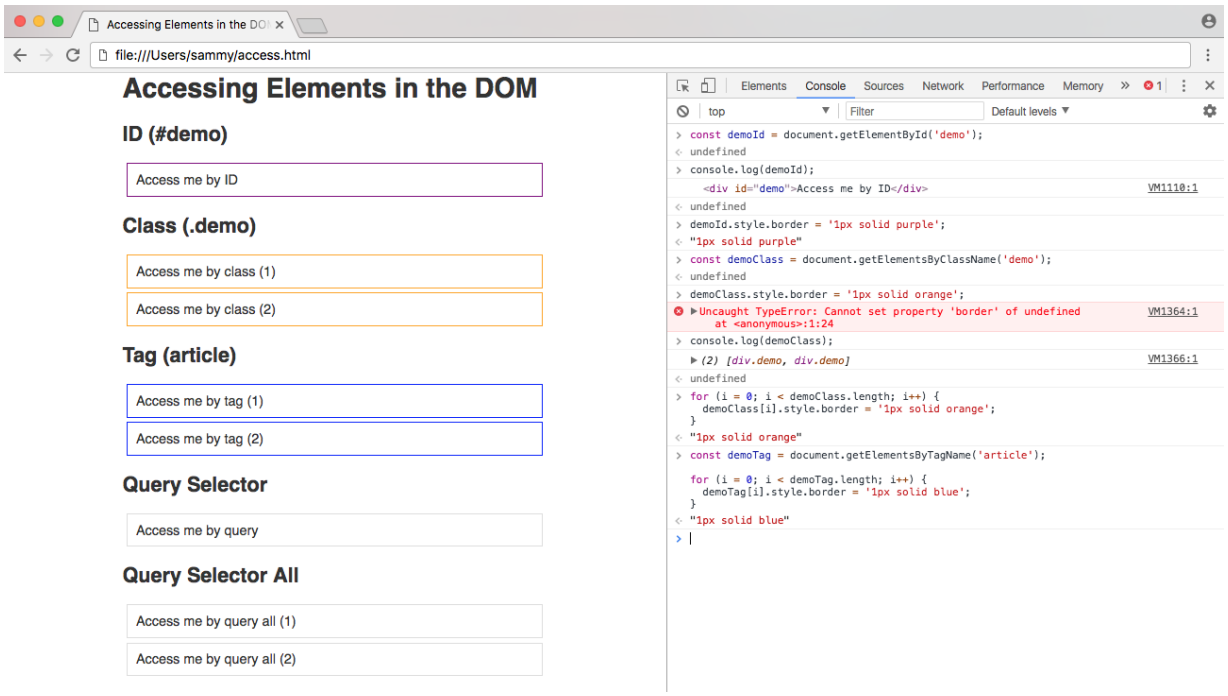
For our tag example, we're using `article` elements.

```
<article>Access me by tag (1)</article>
<article>Access me by tag (2)</article>
```

Just like accessing an element by its class, `getElementsByTagName()` will return an array-like object of elements, and we can modify every tag in the document with a `for` loop.

```
const demoTag =
document.getElementsByTagName('article');

for (i = 0; i < demoTag.length; i++) {
  demoTag[i].style.border = '1px solid blue';
}
```

Upon running the code, the live page will be modified like so:



Browser rendering of tag element styling

The loop changed the `border` property of all `article` elements to `blue`.

Query Selectors

If you have any experience with the jQuery API, you may be familiar with jQuery's method of accessing the DOM with CSS selectors.

```
$('#demo'); // returns the demo ID element in jQuery
```

We can do the same in plain JavaScript with the `querySelector()` and `querySelectorAll()` methods.

```
document.querySelector();
document.querySelectorAll();
```

To access a single element, we will use the querySelector() method. In our HTML file, we have a demo-query element

```
<div id="demo-query">Access me by query</div>
```

The selector for an id attribute is the hash symbol (#). We can assign the element with the demo-query id to the demoQuery variable.

```
const demoQuery = document.querySelector('#demo-query');
```

In the case of a selector with multiple elements, such as a class or a tag, querySelector() will return the first element that matches the query. We can use the querySelectorAll() method to collect all the elements that match a specific query.

In our example file, we have two elements with the demo-query-all class applied to them.

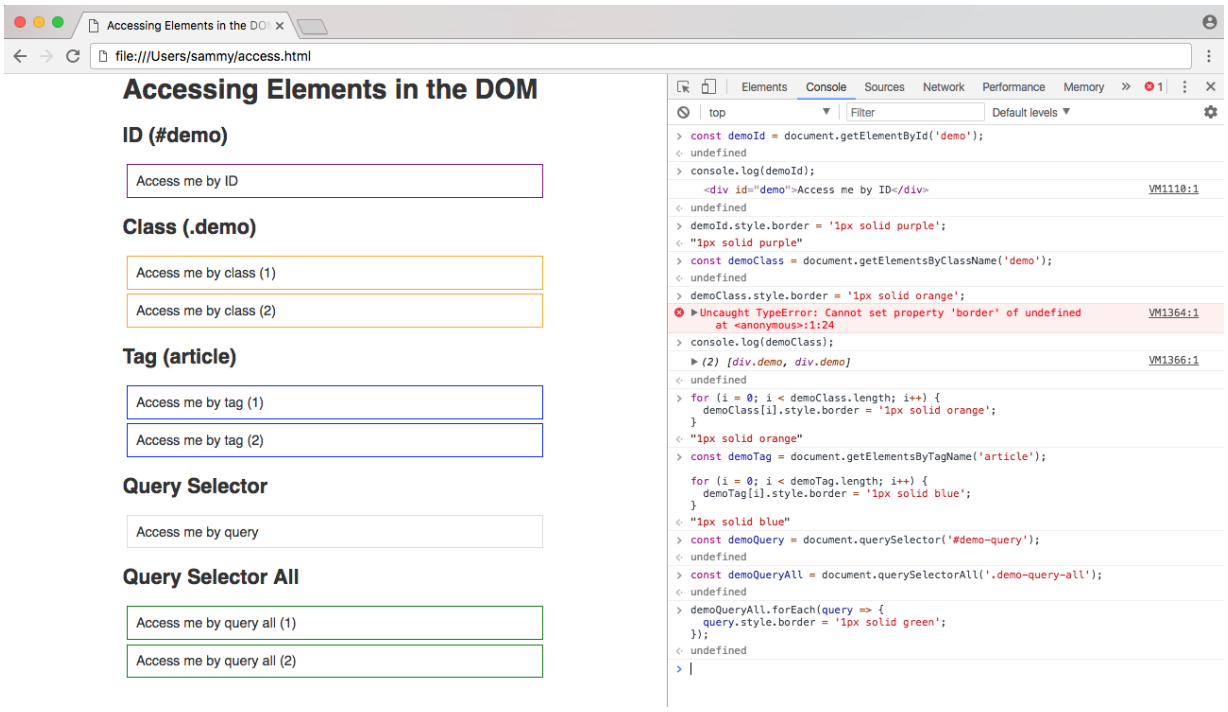
```
<div class="demo-query-all">Access me by query all (1)</div>
<div class="demo-query-all">Access me by query all (2)</div>
```

The selector for a class attribute is a period or full stop (.), so we can access the class with .demo-query-all.

```
const demoQueryAll =
document.querySelectorAll('.demo-query-all');
```

Using the `forEach()` method, we can apply the color `green` to the `border` property of all matching elements.

```
demoQueryAll.forEach(query => {
  query.style.border = '1px solid green';
});
```



Browser rendering of querySelector() styling

With `querySelector()`, comma-separated values function as an OR operator. For example, `querySelector('div, article')` will match `div` or `article`, whichever appears first in the document. With `querySelectorAll()`, comma-separated values function as an AND

operator, and `querySelectorAll('div, article')` will match all `div` and `article` values in the document.

Using the query selector methods is extremely powerful, as you can access any element or group of elements in the DOM the same way you would in a CSS file. For a complete list of selectors, review CSS Selectors on the Mozilla Developer Network.

Complete JavaScript Code

Below is the complete script of the work we did above. You can use it to access all the elements on our example page. Save the file as `access.js` and load it in to the HTML file right before the closing `body` tag.

access.js

```
// Assign all elements
const demoId = document.getElementById('demo');
const demoClass =
document.getElementsByClassName('demo');
const demoTag =
document.getElementsByTagName('article');
const demoQuery = document.querySelector('#demo-
query');
const demoQueryAll =
document.querySelectorAll('.demo-query-all');

// Change border of ID demo to purple
demoId.style.border = '1px solid purple';

// Change border of class demo to orange
for (i = 0; i < demoClass.length; i++) {
  demoClass[i].style.border = '1px solid orange';
}

// Change border of tag demo to blue
for (i = 0; i < demoTag.length; i++) {
  demoTag[i].style.border = '1px solid blue';
}

// Change border of ID demo-query to red
demoQuery.style.border = '1px solid red';
```

```
// Change border of class query-all to green
demoQueryAll.forEach(query => {
  query.style.border = '1px solid green';
});
```

Your final HTML file will look like this:

access.html

```html
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Accessing Elements in the DOM</title>

  <style>
    html { font-family: sans-serif; color: #333; }
    body { max-width: 500px; margin: 0 auto; padding: 0 15px; }
    div, article { padding: 10px; margin: 5px; border: 1px solid #dedede; }
  </style>

</head>

<body>

  <h1>Accessing Elements in the DOM</h1>

  <h2>ID (#demo)</h2>
  <div id="demo">Access me by ID</div>
```

```
  <h2>Class (.demo)</h2>
  <div class="demo">Access me by class (1)</div>
  <div class="demo">Access me by class (2)</div>

  <h2>Tag (article)</h2>
  <article>Access me by tag (1)</article>
  <article>Access me by tag (2)</article>

  <h2>Query Selector</h2>
  <div id="demo-query">Access me by query</div>

  <h2>Query Selector All</h2>
  <div class="demo-query-all">Access me by query
all (1)</div>
  <div class="demo-query-all">Access me by query
all (2)</div>

  <script src="access.js"></script>

</body>

</html>
```

You can continue to work on these template files to make additional changes by accessing HTML elements.

Conclusion

In this tutorial, we went over 5 ways to access HTML elements in the DOM — by ID, by class, by HTML tag name, and by selector. The method you will use to get an element or group of elements will depend on browser support and how many elements you will be manipulating. You should now feel confident to access any HTML element in a document with JavaScript through the DOM.

How To Traverse the DOM

Written by Tania Rascia

The previous tutorial in this series, How to Access Elements in the DOM, covers how to use the built-in methods of the `document` object to access HTML elements by ID, class, tag name, and query selectors. We know that the DOM is structured as a tree of nodes with the `document` node at the root and every other node (including elements, comments, and text nodes) as the various branches.

Often, you will want to move through the DOM without specifying each and every element beforehand. Learning how to navigate up and down the DOM tree and move from branch to branch is essential to understanding how to work with JavaScript and HTML.

In this tutorial, we will go over how to traverse the DOM (also known as walking or navigating the DOM) with parent, child, and sibling properties.

Setup

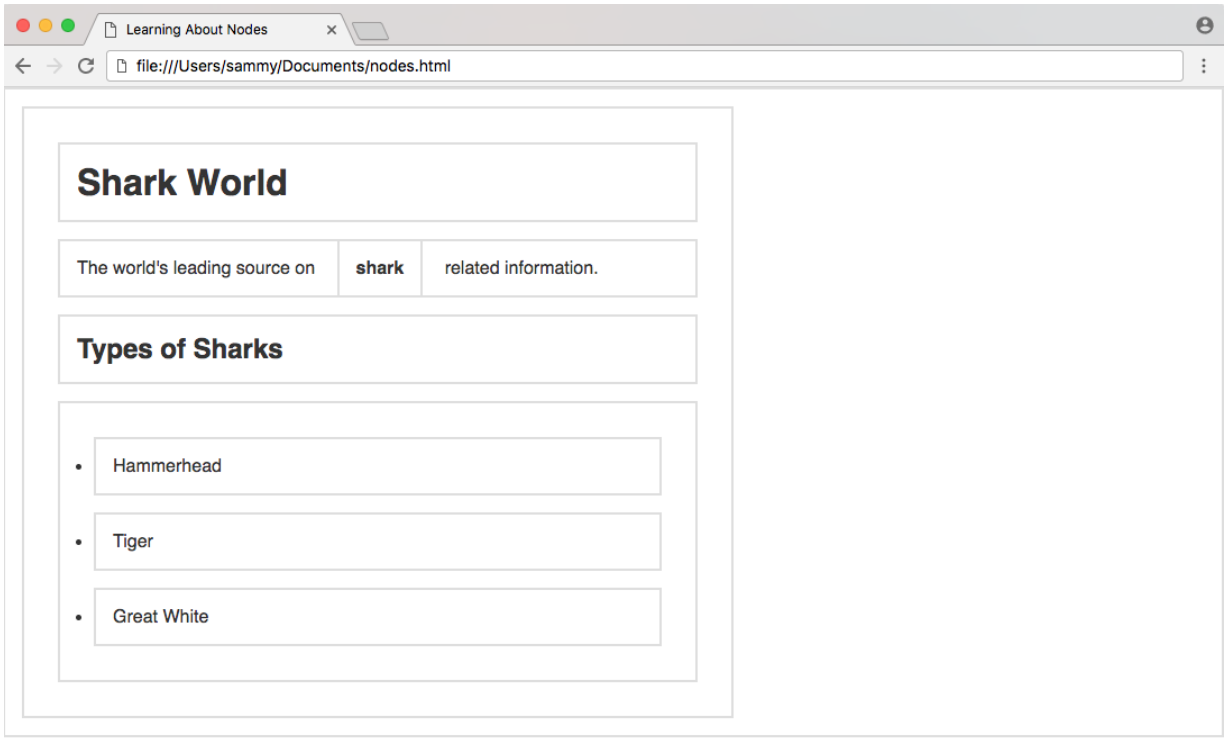
To begin, we will create a new file called `nodes.html` comprised of the following code.

nodes.html

```html
<!DOCTYPE html>
<html>

<head>
  <title>Learning About Nodes</title>

  <style>
    * { border: 2px solid #dedede; padding: 15px;
margin: 15px; }
    html { margin: 0; padding: 0; }
    body { max-width: 600px; font-family: sans-
serif; color: #333; }
  </style>
</head>

<body>
  <h1>Shark World</h1>
  <p>The world's leading source on
<strong>shark</strong> related information.</p>
  <h2>Types of Sharks</h2>
  <ul>
    <li>Hammerhead</li>
    <li>Tiger</li>
    <li>Great White</li>
  </ul>
</body>
```

```
<script>
  const h1 = document.getElementsByTagName('h1')
[0];
  const p = document.getElementsByTagName('p')[0];
  const ul = document.getElementsByTagName('ul')
[0];
</script>

</html>
```

When we load the file in a web browser, we'll see rendering that looks like the following screenshot.



nodes.html page

In this example website, we have an HTML document with a few elements. Some basic CSS has been added in a `style` tag to make each element obviously visible, and a few variables have been created in the `script` for ease of access of a few elements. Since there is only one of each `h1`, `p`, and `ul`, we can access the first index on each respective `getElementsByTagName` property.

Root Nodes

The `document` object is the root of every node in the DOM. This object is actually a property of the `window` object, which is the global, top-level

object representing a tab in the browser. The `window` object has access to such information as the toolbar, height and width of the window, prompts, and alerts. The `document` consists of what is inside of the inner `window`.

Below is a chart consisting of the root elements that every document will contain. Even if a blank HTML file is loaded into a browser, these three nodes will be added and parsed into the DOM.

Property	Node	Node Type
`document`	`#document`	`DOCUMENT_NODE`
`document.documentElement`	`html`	`ELEMENT_NODE`
`document.head`	`head`	`ELEMENT_NODE`
`document.body`	`body`	`ELEMENT_NODE`

Since the `html`, `head`, and `body` elements are so common, they have their own properties on the `document`.

Open the Console in DevTools and test each of these four properties by submitting them and viewing the output. You can also test `h1`, `p`, and `ul` which will return the elements due to the variables we added in the `script` tag.

Parent Nodes

The nodes in the DOM are referred to as parents, children, and siblings, depending on their relation to other nodes. The parent of any node is the node that is one level above it, or closer to the `document` in the DOM

hierarchy. There are two properties to get the parent — `parentNode` and `parentElement`.

Property	Gets
`parentNode`	Parent Node
`parentElement`	Parent Element Node

In our `nodes.html` example:

- `html` is the parent of `head`, `body`, and `script`.
- `body` is the parent of `h1`, `h2`, `p` and `ul`, but not `li`, since `li` is two levels down from `body`.

We can test what the parent of our `p` element is with the `parentNode` property. This `p` variable comes from our custom `document.getElementsByTagName('p')[0]` declaration.

```
p.parentNode;
```

Output

```
▶ <body>...</body>
```

The parent of `p` is `body`, but how can we get the grandparent, which is two levels above? We can do so by chaining properties together.

```
p.parentNode.parentNode;
```

Output

```
▶ <html>...</html>
```

Using `parentNode` twice, we retrieved the grandparent of `p`.

There are properties to retrieve the parent of a node, but only one small difference between them, as demonstrated in this snippet below.

```
// Assign html object to html variable
const html = document.documentElement;

console.log(html.parentNode); // > #document
console.log(html.parentElement); // > null
```

The parent of almost any node is an element node, as text and comments cannot be parents to other nodes. However, the parent of `html` is a document node, so `parentElement` returns `null`. Generally, `parentNode` is more commonly used when traversing the DOM.

Children Nodes

The children of a node are the nodes that are one level below it. Any nodes beyond one level of nesting are usually referred to as descendants.

Property	Gets
childNodes	Child Nodes
firstChild	First Child Node
lastChild	Last Child Node
children	Element Child Nodes
firstElementChild	First Child Element Node
lastElementChild	Last Child Element Node

The `childNodes` property will return a live list of every child of a node. You might expect the `ul` element to get three `li` elements. Let's test what it retrieves.

```
ul.childNodes;
```

Output

```
▶ (7) [text, li, text, li, text, li, text]
```

In addition to the three `li` elements, it also gets four text nodes. This is because we wrote our own HTML (it was not generated by JavaScript) and the indentation between elements is counted in the DOM as text nodes. This is not intuitive, as the Elements tab of DevTools strips out white space nodes.

If we attempted to change the background color of the first child node using the `firstChild` property, it would fail because the first node is text.

```
ul.firstChild.style.background = 'yellow';
[secondary_label Output]
Uncaught TypeError: Cannot set property
'background' of undefined
```

The `children`, `firstElementChild` and `lastElementChild` properties exist in these types of situations to retrieve only the element nodes. `ul.children` will only return the three `li` elements.

Using `firstElementChild`, we can change the background color of the first `li` in the `ul`.

```
ul.firstElementChild.style.background = 'yellow';
```